

Academic Platform

Phased Build Plan

Solo dev · 1 month · 2026-06-07

Academic Platform — Phased Build Plan

Solo dev · 1 month · Next.js 16 monolith

Stack (locked)

Layer	Choice
Framework	Next.js 16 (App Router, TS)
DB	PostgreSQL + Drizzle ORM
Auth	Auth.js (credentials + bcrypt + JWT)
AI	Groq — Llama 3.3 70B + Whisper Turbo STT
TTS (read-aloud + interview)	Browser SpeechSynthesis (free)
Code sandbox	Judge0 (RapidAPI → self-host VPS)
Storage	Cloudflare R2 (zero egress)
Hosting	Hetzner VPS + Coolify
Cache	Postgres <code>generated_content</code> table (persistent)

Excluded: NestJS microservices, Qdrant/RAG, Redis, Kubernetes, video.

Hierarchy

Section (school / intermediate / college / postgrad / professional)

└ Provider (board for school+inter · university for college+pg)

└ Grade (Class 5 · B.Tech Year 2)

└ Subject (is_coding flag unlocks coding module)

└ Chapter

└ Topic

└ Content (rich text + PDF + read-aloud TTS)

└ Mock Test (single-topic · human bank + AI gen)

└ Mock Interview (multi-chapter + multi-topic · voice + text)

└ Coding (CS only · MCQ + Judge0 sandbox)

Roles

- **Super Admin** — global. CRUD all providers/grades/subjects/topics, all content, all questions. Approve students. Full analytics.
 - **Admin** — scoped to assigned providers. CRUD curriculum + content + questions inside scope.
 - **Student** — signup → `pending` → admin manual approve → dashboard scoped to their board.
-

Phase 0 — Setup DONE

Built:

- Next.js 16 + TS + Tailwind v4 + shadcn/ui ready
- Drizzle + Postgres (Docker, port 5433)
- 25 tables migrated (`drizzle/0000_*.sql` applied)
- `/api/health` returns `database: up`
- Module folder layout
(`src/modules/{auth,content,assessment,interview,coding,admin}`)
- Git: `cf805f3` scaffold · `dc0130a` multi-topic interview + port fix

Gate:  Server live `http://localhost:3000` · health green · all tables present.

Phase 1 — Auth + RBAC + Approval (3–4 days)

Build:

- Auth.js credentials provider, bcrypt password hashing
- Signup form: name, email, phone, country, state → cascading section → provider → grade → submit (`status=pending`)
- Login + session cookie
- Middleware: block `pending` from dashboard · gate `/admin/*` by role
- Super-admin approval queue: list pending, approve/reject (writes `audit_log`)
- Seed: 1 super-admin + 5 sections + 1 sample board (CBSE) + Class 5

Tables touched: `users` , `admin_scope` , `sections` , `providers` , `grades` , `audit_log`

Gate: Pending user blocked at dashboard · approved student reaches empty dashboard · admin approves from `/admin/approvals` .

Phase 2 — Curriculum CRUD (4–5 days)

Build:

- Tree UI: providers → grades → subjects → chapters → topics
- `is_coding` toggle on subject creation
- Admin scoped to their providers via `admin_scope` ; super-admin sees all
- Inline create/edit/delete + drag-sort

Tables touched: `providers` , `grades` , `subjects` , `chapters` , `topics`

Gate: Admin builds **CBSE** → **Class 5** → **English** → **Tenses chapter** → **"Present Tense" topic** fully from UI.

Phase 3 — Content + Student Learning + TTS (4–5 days)

Build:

- Topic content editor (rich text → `topic_content.body_html`)
- PDF/image upload → R2 (signed URLs, presigned PUT for uploads)
- Student dashboard board-scoped via central `withBoardScope()` query helper
- Topic viewer: rich content + PDF preview (`react-pdf` / `iframe`)
- Browser SpeechSynthesis read-aloud: play/pause/speed selector
- Mark `progress.content_viewed = true` on view

Tables touched: `topic_content` , `content_assets` , `progress`

Gate: Student logs in, navigates to Tenses, reads content, hears TTS, sees PDF — sees **only their board's** content (cross-board leak test = blocked).

Phase 4 — Mock Test + AI Gen + Cache (4–5 days)

Build:

- Admin question bank CRUD (MCQ with options + subjective)
- "AI-generate questions" button → Groq Llama → write to `generated_content` cache + insert into `questions`
- Student test flow: load (cache hit = instant), answer, submit, auto-score MCQ, store attempt + answers
- Update `progress.test_best` on submit
- Difficulty filter (easy/medium/hard)

Tables touched: `questions` , `question_options` , `test_attempts` , `test_answers` , `generated_content`

Cache key: `sha256(section|provider|grade|subject|topic|mock_test|difficulty)`

Gate: Human-authored + AI-generated tests both work · 2nd student on same topic+difficulty hits cache (no LLM call).

Phase 5 — Mock Interview Voice + Text (4–5 days)

Build:

- Session start UI: pick subject → multi-select chapters → multi-select topics across chapters → start
- Text mode first (chat UI), then voice mode
- Voice flow: record audio (MediaRecorder) → POST → Groq Whisper STT → transcript → Llama scores + generates follow-up → browser TTS speaks next question
- Cache key includes **sorted topic_ids set** + difficulty + mode
- Store session, per-answer transcript + score, overall feedback

Tables touched: `interview_sessions` , `interview_session_topics` , `interview_questions` , `interview_answers` , `generated_content`

Gate: Full voice interview across 3 selected topics from 2 chapters · speak → transcript → score → next question spoken aloud · `progress.interview_best` updates.

Phase 6 — Coding Module + Sandbox (4–5 days)

Build:

- Coding module visible only when `subject.is_coding = true`
- Admin authors coding questions + hidden + sample test cases
- Student code editor: Monaco · language picker · starter code per language
- Submit → Judge0 (RapidAPI key) → run test cases → weight-graded score → store submission + Judge0 token + per-case results
- MCQ coding questions also supported
- Async status polling for long-running submissions

Tables touched: `coding_questions` , `coding_test_cases` , `coding_submissions`

Security non-negotiables (Judge0 enforces): network off, CPU/mem/time/PID caps, ephemeral isolation, no host disk access.

Gate: Student writes Python, runs against test cases, sees accepted/wrong + per-case stdout/stderr · `progress.coding_solved` increments.

Phase 7 — Analytics + Hardening + Deploy (3–4 days)

Build:

- Super-admin analytics drill-down: section → provider → grade → subject → counts (students, attempts) + avg scores (test/interview/coding)
- Rate-limit AI endpoints (per-user + global)
- Audit log review page
- Backups: `pg_dump` cron + R2 upload
- Swap RapidAPI Judge0 → self-hosted Judge0 on VPS
- Error monitoring (Sentry free tier)
- Deploy Hetzner CPX21 + Coolify

Tables touched: reads `progress`, `*_attempts`, `*_submissions`; no new tables.

Gate: Super-admin clicks School → CBSE → Class 5 → English → sees real student counts + avg scores · backups run nightly · sandbox self-hosted.

Sequencing Rules

- **Critical path:** P0 → P1 → P2 → P3 must finish in order. Tree + content gate everything else.
 - **P4, P5, P6 priority-pickable:** can reorder by importance. If time slips, ship **P0–P4 as v1**; push P5/P6 to weeks 5–6.
 - **Defer to v2:** cloud TTS, payments, notifications, parent role, certifications, RAG cross-syllabus tutor, video interview, richer dashboards.
 - **Never cut:**
 - Sandbox isolation (P6) — security
 - `withBoardScope()` helper (P3) — one leak = wrong-board content exposed
-

Per-Phase Loop (workflow)

1. Claude writes/edits files in D:\academic-platform
2. You: pnpm install (if deps added) → pnpm dev
3. Click around • test in browser
4. Broken? paste error → Claude fixes
5. Phase gate green → next phase

Accounts Needed (when, not now)

Phase	Service	Cost
P3	Cloudflare R2	Free 10 GB
P4, P5	Groq API key	Free 14,400 req/day
P6	Judge0 RapidAPI	Free tier
P7	Hetzner VPS	~€8/mo
P7	Sentry	Free 5k events/mo

Local dev (P0–P6) = \$0 (Docker Postgres).

Status as of 2026-06-07

- **Phase 0:**  DONE
- **Phase 1:** next — auth + signup + approval
- **Commits:** cf805f3 , dc0130a on master
- **DB:** Postgres 16 in Docker, port **5433** (5432 taken by other project)
- **Dev URL:** http://localhost:3000